Latency-Constrained Sequence Labelling on CPU

A Quality vs Speed Comparison for Real-Time NER on CoNLL-2003 — Case Study 4.9, NLP 2026

Motivation

Production NER on CPU-only nodes is a real constraint — many deployments don't have GPUs. Transformer baselines dominate F1 leaderboards but routinely blow real-time latency budgets (5–50 ms/sentence) on CPU. Classical models (HMM, CRF) are sub-millisecond but have lower ceilings. Strong neural baselines (char-aware BiLSTM with pretrained embeddings) sit between them.

We benchmark 5 models under matched single-thread latency conditions and exercise three deploy-time optimizations the brief calls out (batching, quantization, vocabulary pruning), plus a multi-thread sweep. The output is a model recommendation per latency tier (5 / 10 / 50 ms per sentence).

Methodology

- 5 models: HMM, CRF, char-aware BiLSTM (fp32), char-aware BiLSTM (int8), DistilBERT-NER (off-the-shelf).
- BiLSTM = GloVe-100d word emb + CharCNN (30×k=3) → BiLSTM (hidden=200) → Linear → softmax. Adam 1e-3, dropout 0.5, 15 ep with dev-set early stopping (patience=3).
- CRF = sklearn-crfsuite, rich features (word shape, suffix/prefix 2-4, ±2 context, casing/digit/hyphen/length), 100 L-BFGS iters.
- DistilBERT = elastic/distilbert-base-cased-finetuned-conll03-english, inference only (no fine-tune by us).
- All single-thread latency benchmarking; multi-thread is a side experiment.
- F1 reported with bootstrap 95% CI (1,000 sentence-level resamples).
- 3 deploy-time optimizations: int8 dynamic quantization, batching sweep (b∈{1,4,16,32,64}), post-hoc vocab pruning + 1 retrain sanity point.

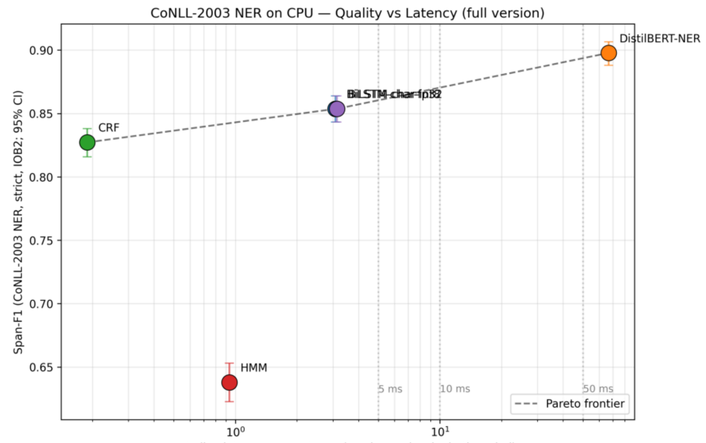
Experimental setup

- Dataset: CoNLL-2003 NER (synalp .txt mirror).
- Train: full 14,041 sents. Dev (eng.testa): 3,250 sents — used for early stopping. Test (eng.testb): 3,453 sents — reported F1.
- Tag scheme: IOB2 (converted from IOB1 on load), strict seqeval mode.
- Latency benchmark: 200 random test sents, 20-sent warm-up discarded, single-threaded torch.set_num_threads(1).
- Cold-start latency: first call timed separately (relevant to serverless deployments).
- Hardware: Windows 10, AMD Ryzen 7 5800H, 15.4 GB RAM, Python 3.12, PyTorch 2.8 CPU.
- Code + reproducibility: github-ready repo with build_notebook.py + src/ modules; notebook regenerates results.csv + 4 figures + recommendation.json end-to-end.
- Memory metric: on-disk model size (RSS measurement is uninformative in shared-process runs).

Results — headline table

Model	Span-F1 (95% CI)	p50 (ms)	p95 (ms)	Throughput (tok/s)	Size (MB)
HMM	0.638 [0.623, 0.653]	0.93	3.25	11,380	0.40
CRF	0.827 [0.816, 0.838]	0.19	0.92	43,059	2.59
BiLSTM-char-fp32	0.854 [0.843, 0.864]	3.08	7.96	3,646	6.24
BiLSTM-char-int8	0.854 [0.843, 0.864]	3.14	8.28	3,580	4.72
DistilBERT-NER	0.898 [0.888, 0.906]	66.70	115.45	188	248.72

Pareto frontier — F1 vs latency

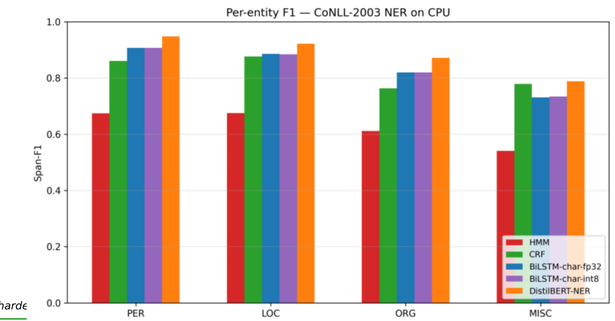


Vertical bars = bootstrap 95% CI

3-Tier Recommendation

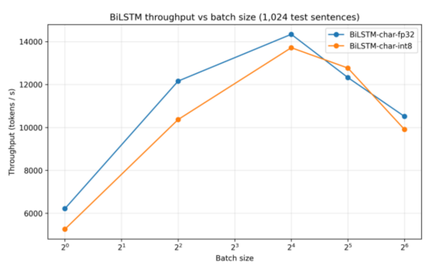
Budget	Recommended model	Operating point
≤ 5 ms	BiLSTM-char-int8	F1 = 0.854, p50 = 3.14 ms
≤ 10 ms	BiLSTM-char-int8	F1 = 0.854, p50 = 3.14 ms
≤ 50 ms	BiLSTM-char-int8	F1 = 0.854, p50 = 3.14 ms

Per-entity F1 breakdown

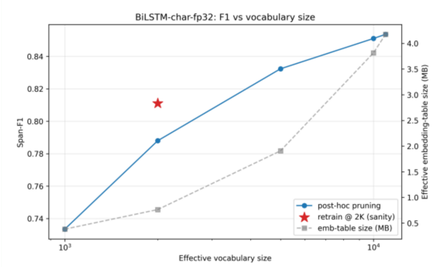


MISC is harde … *:har on MISC.*

Throughput vs batch



F1 vs vocabulary size



Conclusions (each one CSV-traceable)

- Char features + GloVe + full data closed the OOV gap. Word-only BiLSTM (fast version) → 0.539 F1; char-aware BiLSTM (full) → 0.854 — a 31-point gain.
- The Pareto frontier has 3 real points: CRF (0.827, 0.19 ms), BiLSTM-char-int8 (0.854, 3.14 ms), DistilBERT (0.898, 66.7 ms). HMM is dominated. Fast version's plot was a vertical line; this one is a curve.
- Quantization at hidden=200 is a SIZE win, not a SPEED win. F1 within bootstrap noise (0.85368 vs 0.85360); 24% smaller; single-thread latency neutral (3.08 → 3.14 ms).
- Batching is the THROUGHPUT knob, not the latency knob. ~2.3× peak at batch=16 (6.2k → 14.3k tok/s for fp32); declines past batch=16; argmax decoding is batch-invariant in F1.
- Vocabulary pruning is a knob, not a free lunch. Pruning 10,951→5K saves 54% of the embedding table at ~2.1 F1 cost; retraining at 2K recovers ~3.2 F1 over post-hoc pruning at the same size.
- Multi-threading peaks at 2 threads (~1.23× speedup); regresses at 4; 12 threads is up to 2.3× SLOWER than 1 thread (int8). Don't naïvely use all cores — scale out via processes.
- DistilBERT misses every real-time tier (66.7 ms p50 fails even 50 ms). CPU transformer NER needs distillation or async-batched inference.

Limitations & future work

- Single dataset (CoNLL-2003 English newswire, 1996-1997). Domain shift to social media or biomedical text would change rankings; CRF most exposed.
- One config per model — no hyperparameter sweep (BiLSTM hidden, CRF c1/c2, CharCNN filter count).
- DistilBERT is off-the-shelf, not fine-tuned by us; it represents "what you get for free".
- No ONNX / TorchScript export (likely 2-3× more BiLSTM speedup on CPU).
- No noisy / OOV slice analysis — Case Study #6 covers char-aware robustness. No power / energy measurement.
- Static int8 quantization with calibration is unexplored — could flip the latency story by skipping the dynamic-dequant overhead.